

POLITECNICO
MILANO 1863

HOP-PAAL: A Formal Frogger Journey

Formal Methods for Software Engineering
A.Y. 2025-2026

Dario Cambi, Giorgio Ciraci and Daniele Crippa

Politecnico di Milano
dario.cambi@mail.polimi.it
giorgio.ciraci@mail.polimi.it
daniele5.crippa@mail.polimi.it

Abstract

UPPAAL is a tool for modeling, simulating, and verifying real-time systems as networks of Timed Automata. In this report, we present the modeling and formal verification of a simplified version of Frogger, a classic arcade game. Both symbolic and a stochastic version of the model are developed and analyzed.

Contents

1 Introduction **1**
1.1 Assumptions and further clarifications 1

2 Model Description **2**
2.1 Overall Design Choices 2
2.2 Frogger 3
2.3 Vehicle Lane 4
2.4 Floating Object Lane 4
2.5 Diving Turtles Manager 5
2.6 Channels 5
2.7 Stochastic Variation 6
2.7.1 Frogger 6
2.7.2 Truck Lane 7

3 Properties Verification **8**
3.1 Symbolic Verification 8
3.2 Stochastic Analysis 9
3.2.1 Completing the first level with at least 200 points 9
3.2.2 Context-Aware Movement Policy 9
3.2.3 Context-Aware Policy Analysis 10

4 Conclusion **12**

References

[1] UPPAAL Documentation. `https://docs.uppaal.org/`.

1 Introduction

Frogger is a classic arcade video game released in 1981, one of the most iconic titles of its era. The primary goal of the game is to successfully guide a frog into a **home bay** as many times as possible, requiring the player to cross a **busy highway** and a **dangerous river**, avoiding moving cars and jumping on logs and turtles to avoid falling into the water.

Formal verification is a technique used to **prove or disprove the correctness of a system** with respect to a formal specification. While commonly applied to **hardware and safety-critical software**, it can also be used to model and verify the behavior of more unconventional systems, such as video games, which are rarely formally defined.

UPPAAL is a tool designed for **modeling, simulating, and verifying real-time systems** using networks of **Timed Automata**. It provides a graphical interface alongside a powerful **query language** that allows the user to formally check properties on the system.

In this project, UPPAAL is applied to **model and formally verify** a simplified version of Frogger, covering all its **core mechanics** as defined in the specifications. The goal is to demonstrate that the model **behaves correctly under covered conditions** and to **verify some given properties**, using both a **vanilla** and a **stochastic** version of the automata network.

1.1 Assumptions and further clarifications

The following assumptions are made on the specifications:

- One time unit in UPPAAL corresponds to 0.1 seconds
- Between any two consecutive moves of Frogger, at least 1 time unit must elapse; this prevents unrealistic Zeno behaviors
- The game timer is assumed to start when Frogger respawns, ignoring the respawn cooldown. As a consequence, the last playable run starts with a game timer of 2 seconds, during which Frogger is bound to die, since the respawn
- Frogger cannot move outside the visible grid.
- Levels are indexed starting from one.

After further context clarifications with the professor we also defined the following edge cases or ambiguous situations:

- If Frogger attempts to reach an invalid cell, i.e. a green cell, an occupied home bay or a cell out of the map, the move is simply blocked rather than resulting in its death.

2 Model Description

The model is based on 4 primary components:

- *Frogger*
- *Vehicle Lane*
- *Floating Lane*
- *Diving Turtles Manager*

Frogger

The protagonist of the game, it starts with 5 lives and must reach all 5 home bays to complete a level. The frog dies upon collision with a vehicle, contact with open water, or expiration of the time limit.

Vehicle Lane

Models a road lane populated by moving vehicles of varying speed and size. Each lane defines a fixed direction of travel, and a collision with the frog results in the loss of a life.



Figure 1: Vehicle lane graphic from the game

```
{S,S,S,S,V,S,S,S,S,S,S,S,S,V,S,S,S,S,S}, // lane 1  - orange cars
{S,S,V,S,S,S,S,S,S,S,S,V,S,S,S,S,S,S,S}, // lane 2  - bulldozers
{S,S,S,V,S,S,S,S,S,V,S,S,S,S,V,S,S,S,V,S,S}, // lane 3  - green cars
{S,S,V,S,S,S,S,S,S,S,V,S,S,S,S,S,S,S,S}, // lane 4  - purple cars
{S,S,S,V,V,S,S,S,S,S,S,S,S,S,V,V,S,S,S,S}, // lane 5  - trucks
```

Figure 2: Vehicle lane matrix representation

Floating Lane

Models a river lane composed of floating objects, such as logs and turtles. When the frog stands on a floating object, it is carried along with it. Landing on an empty water cell results in immediate death.

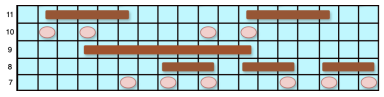


Figure 3: Floating lane graphic from the game

```
{W,W,W,W,W,T,T,T,T,T,W,W,W,D,D,D,D,D,D,W,W}, // lane 7  - turtles/diving turtles
{W,W,W,W,W,W,W,L,L,L,W,L,L,L,W,L,L,L,W,W}, // lane 8  - logs
{W,W,L,L,L,L,L,L,L,L,L,W,W,W,W,W,W,W,W,W}, // lane 9  - long log
{W,T,T,T,W,W,W,W,W,D,D,D,W,W,W,W,W,W,W,W}, // lane 10 - turtles/diving turtles
{W,L,L,L,L,L,W,W,W,L,L,L,L,L,W,W,W,W,W,W}, // lane 11 - logs
```

Figure 4: Floating lane matrix representation

Diving Turtles Manager

Manages the submersion cycle of diving turtles, cycling through the states *Emerged*, *Diving*, *Submerged*, and *Emerging*. When turtles submerge, a broadcast signal notifies the Floating Lane, which treats the occupied cell as open water, causing the frog to die if standing on it.

2.1 Overall Design Choices

Before describing each template in detail, we summarise the main design choices that shape the whole model and that are shared across its components.

Game Map

The game map is encoded as a fixed matrix of size `GRID_H`×`LANE_W`, where each cell holds its initial logical content. A central choice is that moving objects are *never* physically rewritten in this matrix: their motion is represented by a single per-lane shift variable inside the lane templates. This keeps the representation compact and avoids continuously updating a large array, which would otherwise enlarge the state space.

Lane Abstraction

Roads and rivers are both modelled as lanes, since they share most of their logic; within each lane, all objects move at the same speed. This avoids modelling each element separately and prevents the state space from exploding. The motion of objects is not rewritten in the map matrix but represented by a single per-lane shift variable, keeping the representation compact.

The logical lane width `LANE_W` is intentionally larger than the visible grid `GRID_W` (20 against 18): the two extra off-screen cells model the re-entry delay required by the specification, so that an object leaving the map does not reappear immediately on the other side. For the same reason, `frog_x` ranges over the full `LANE_W`, allowing Frogger to be carried slightly off-screen while riding a floating object.

Lanes Handle Frogger's Death

The decision to kill Frogger is delegated to the lanes rather than to the frog itself. Each time Frogger moves, it broadcasts this event on a dedicated channel; the broadcast is captured only by the lane on which Frogger is currently located, which then checks whether the new position results in a collision or an unsafe cell. Since each lane knows the global position of Frogger, it is the natural place to perform this check; when an unsafe condition is detected, the lane requests the death through the `kill_frog` channel, which the **Frogger** automaton consumes by removing a life and respawning. This keeps the collision logic local to each lane and leaves the frog simple and focused on its own movement and life management.

Turtles Management

In the game, all diving turtles follow the same submersion cycle, so this global cycle is delegated to a separate template, in which the shared timing is defined in one place and simply observed by the lanes.

Moreover, the water gaps between turtles of the same group are encoded as turtle cells, since Frogger standing on those cells is equally safe, the two are semantically equivalent.

2.2 Frogger

The frogger template contains the main logic of the frog, including its movement, life management and points assignment.

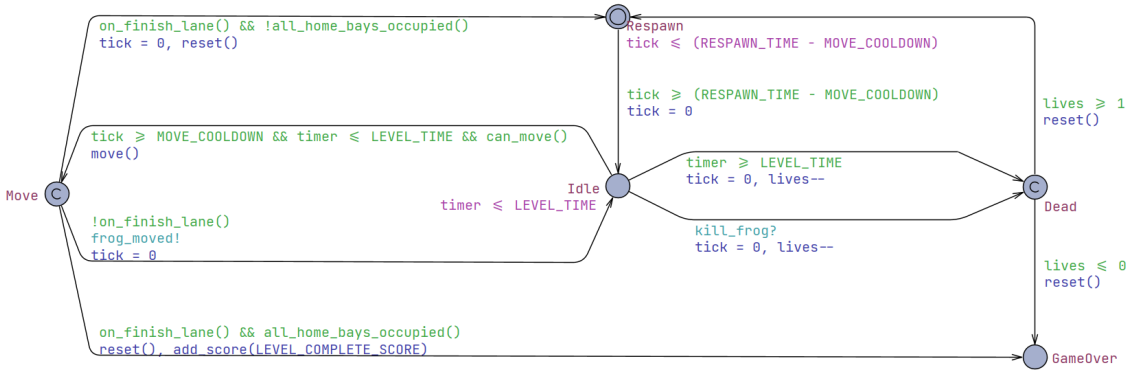


Figure 5: Frogger Template

The design was made around its possible states:

1. **Idle** is the central state where the frog is waiting between actions. Three transitions can fire from here: if the move is legal and the cooldown time is over, the frog can go to the **Move** state; it can be killed either by another template through the `kill_frog` channel or by running out of time, losing a life and sending it to the **Respawn** state.

2. **Move** is a *committed* state in which Frogger enters after a movement command; here, immediately is determined the outcome of its move. If it lands on a finishing lane, it resolves

whether a home bay is reached through; otherwise it simply update the position of Frogger and returns to **Idle**.

3. **Respawn** is reached whenever the frog loses a life or reaches an home bay. From here, if at least one life remains, the frog waits for the respawn timing and then returns to **Idle**. If `lives <= 0`, it transitions to **GameOver** instead.

4. **GameOver**: the final state of the automaton, entered when the frog has no lives left or when the level is completed.

2.3 Vehicle Lane

The vehicle lane template models a single lane of the road, populated by vehicles that move at the same speed and share the same size; both parameters are constants and defined at the instantiation of the template.

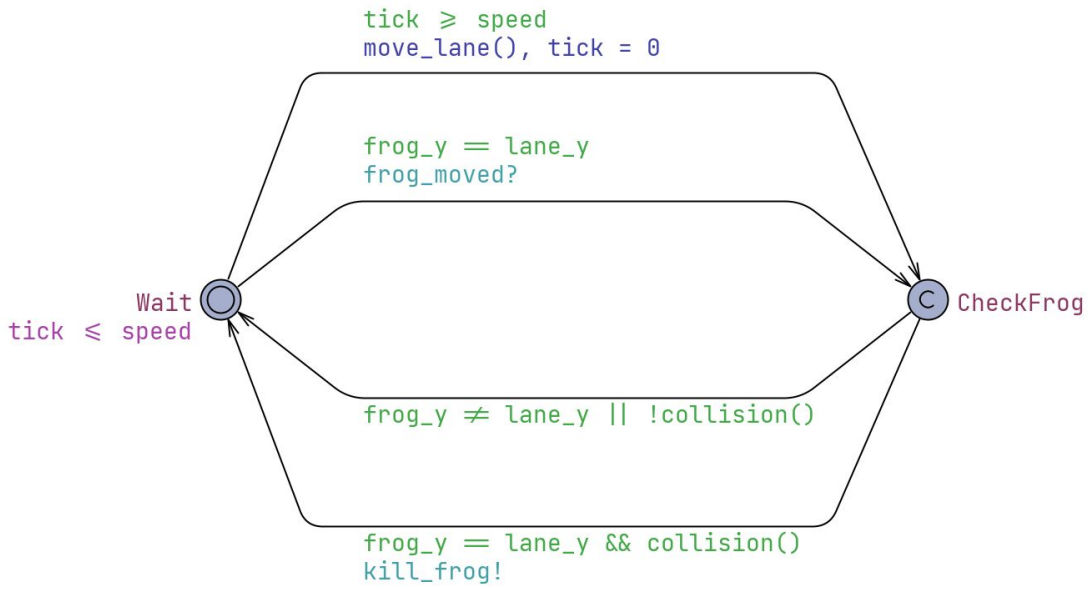


Figure 6: Vehicle Lane Template

The template is realised through two states, **Wait** and **CheckFrog**. In **Wait**, the lane lets time pass until the interal clock reaches the speed; when this happens, it shift the vehicles with the function `move_lane()`, resets the timer, and transitions to **CheckFrog**. This is a *committed* state, so the collision check is performed instantaneously: if a vehicle occupies Frogger's position, it signals the death through the `kill_frog` channel and returns to **Wait**; otherwise it returns to **Wait** without sending any signal.

A second path into **CheckFrog** is triggered when the frog moves: the lane is notified through the `frog_moved` channel, allowing it to verify immediately whether the frog's new position is occupied by a vehicle; if so it signals it through the channel `kill_frog`.

2.4 Floating Object Lane

The floating lane template models a river lane, where the frog can only survive by standing on floating objects, such as logs and turtles.

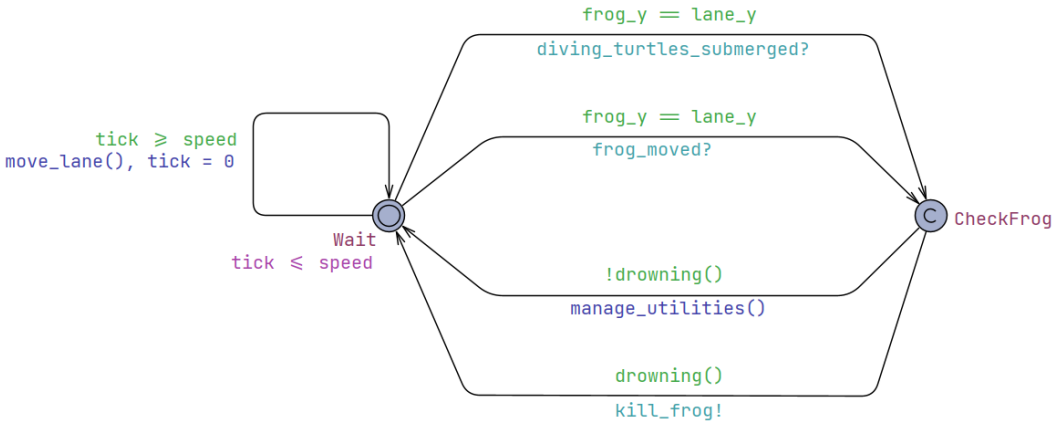


Figure 7: Floating Object Lane Template

The template is structured in two main states, **Wait** and **CheckFrog**. In **Wait**, the lane lets time pass until the internal clock reaches the speed of the lane; when this happens, it shifts the floating objects with the function `move_lane()` and transitions to **CheckFrog**. If the frog is currently standing on a floating object, its global position is updated within the same function, so that it is carried along with the object.

CheckFrog is a *committed* state where the lane verifies whether the frog is safe, and it can be reached through three different paths. The first is the periodic movement just described: after shifting, the lane checks if the frog is drowning with the function `drowning()`, and if it not, it safely returns to **Wait**; otherwise it signals the death through the `kill_frog` channel. The second path is triggered when the frog moves onto the lane, notified through the `frog_moved` channel, allowing the lane to immediately check whether the new position is occupied by a floating object. The third is triggered when the diving turtles submerge, signalled through the `diving_turtles_submerged` channel: if the frog was standing on those turtles, the cell becomes open water and the frog drowns.

2.5 Diving Turtles Manager

The diving turtles manager template models the periodic submersion cycle of the diving turtles through four states: *Emerged*, *Diving*, *Submerged*, and *Emerging*.

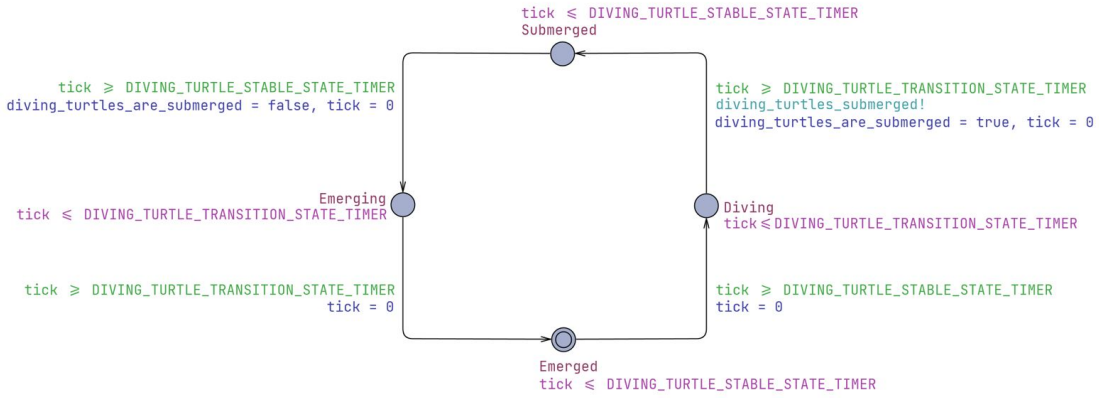


Figure 8: Diving Turtles Manager Template

2.6 Channels

The communication between templates relies on three synchronisation channels, each chosen as broadcast or binary according to how many automata must react to the event:

- `kill_frog` (binary): requests the death of Frogger. It is sent by a **VehicleLane** when a vehicle overlaps the frog, or by a **FloatingLane** when the frog is on unsafe water or on a submerged diving-turtle group. It is received only by the **Frogger** automaton, which removes one life and respawns the frog.
- `frog_moved` (broadcast): announces that Frogger has completed a movement step. It is received by the **VehicleLane** and **FloatingLane** templates, so that the current lane where Frogger is standing can check whether the new position is unsafe; if so it signals the death through the `kill_frog` channel. It is broadcast because a priori is unkown which lane is interested in the message.
- `diving_turtles_submerged` (broadcast): announces that the diving turtles have entered the submerged phase. It is sent by the **Diving Turtle Manager** and received by the **FloatingLane** templates containing diving turtles, which then check whether Frogger is standing on a submerged turtle.

2.7 Stochastic Variation

In the stochastic variant, some of the previous timed automata are modified to implement probabilistic features: Frogger moves in each direction with a different probability weight, while the truck lane moves at a non-deterministic speed.

2.7.1 Frogger

In the deterministic template, once the guard `tick >= MOVE_COOLDOWN` is satisfied, the frog is allowed to leave **Idle** and move in a non-deterministic direction (`UP`, `DOWN`, `LEFT`, `RIGHT`). Here, the choice is resolved through four weighted branches instead, whose normalised weights become the actual probability of each direction.

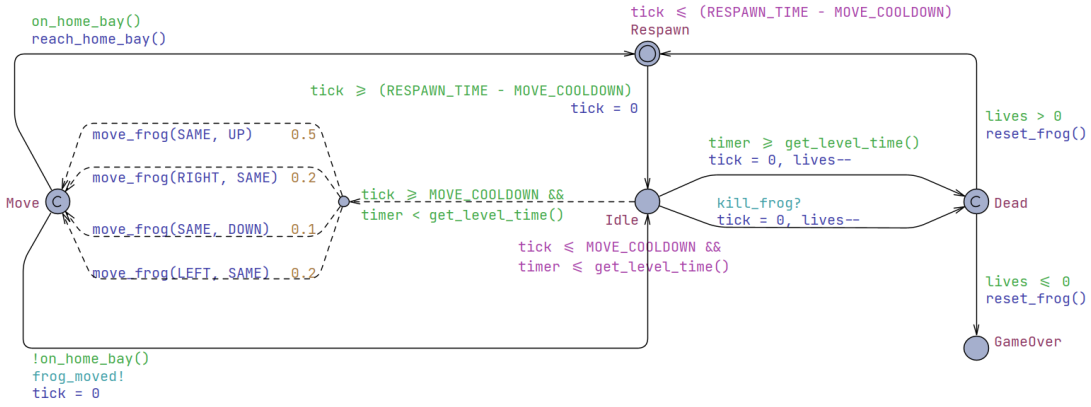


Figure 9: Frogger Template – Stochastic Variation

2.7.2 Truck Lane

In the deterministic template, the lane moved on a fixed rate as soon as the internal clock reached `speed`. Here, **Wait** instead carries a rate of exponential `1:speed`, i.e. $\lambda = 1/\texttt{speed}$: since the location has no invariant, the probability of `move_lane()` firing within t time units is $Pr(t) = 1 - e^{-t/\texttt{speed}}$.

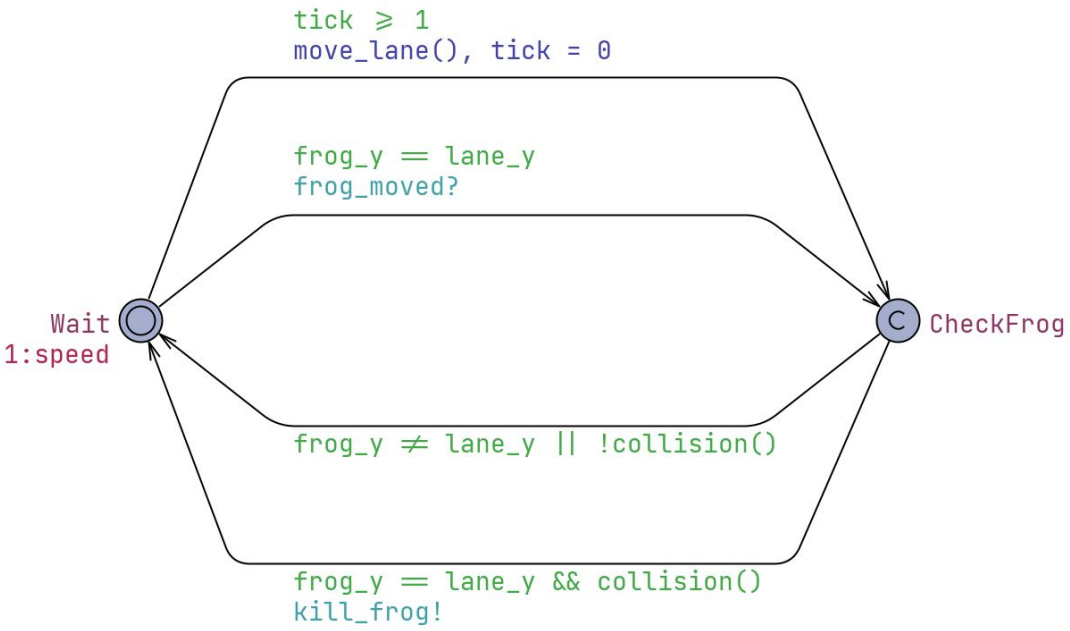


Figure 10: Truck Lane Template – Stochastic Variation

3 Properties Verification

In this seciton the properties specified by the specifications are verified through the UPPAAL verifier, in which the queries can be found. Execution times were derived from benchmarks conducted on a low-cost, consumer-grade laptop.

3.1 Symbolic Verification

The following properties are expressed in TCTL, a simplified version is used in UPPAAL for the formal verification. Each property is briefly commented to explain its meaning with respect to the system behavior.

Absence of deadlock (without Frogger)

$$\text{(I)}: \quad \forall\Box \neg\, deadlock$$

Verified on the system without the Frogger automaton, this property states that along every path the model never reaches a deadlock state, i.e. the lanes and the diving-turtle cycle can always progress.

Execution time: ∼10s

All vehicles and floating objects can only move inside their predefined row

$$\text{(II)}: \quad \forall\Box \left(\forall i \;\; L[i].row = i\right)$$

This property states that each vehicle and each floating object remains within its predefined row and never changes lane. It holds by construction, since the lane templates do not include any transition that would allow an object to move vertically.

Nonetheless, it can still be explicitly modelled as "the lane never changes its row" and verified, in order to confirm that no unexpected behaviour causes an object to leave its lane.

Execution time: ∼5s

Frogger can reach the home bay in position (0,12)–(1,12)

$$\text{(III)}: \quad \exists\Diamond \left(home_bays[0] = true\right)$$

It's saying that exists at least one path that eventually will lead to Frogger reaching the first home bay cell.

Execution time: ∼5s

Frogger can score 17 points reaching the home bay in position (8,12)–(9,12)

$$\text{(IV)}: \quad \exists\Diamond \left(home_bays[2] = true \;\wedge\; score = 17\right)$$

A refinement of the previous property: there exists a path on which Frogger reaches the home bay at $(8, 12)$–$(9, 12)$ while holding exactly 17 points.

Execution time: ∼10s

Frogger can reach a yellow cell in row 6 after the player has lost two lives

$$\text{(V)}: \quad \exists\Diamond \left(frog_y = 6 \;\wedge\; lives = MAX_LIVES - 2\right)$$

It states that there exists a path on which Frogger reaches a yellow cell, the safe lane between the road and the river, after already having lost exactly two lives.

Execution time: ∼10s

Frogger may stand on a log (with five lives) 10 seconds after the game starts

$$\text{(VI)}: \quad \exists\Diamond\left(lives = MAX_LIVES \wedge frog_timer = 100 \wedge frog_on_log\right)$$

There exists a path on which eventually Frogger, without having lost any lives after 10 seconds the game started, may stand on a log. Here to predicate on Frogger's location, as exaustively explained in the project's Documentation, we check the value of the flag *frog_on_log*.

Execution time: ∼2min

Frogger can die riding a diving turtle that transitions to the submerged state

$$\text{(VII)}: \quad \exists\Diamond \left(\mathit{diving_turtles_are_submerged} \wedge \mathit{frog_on_diving_turtle} \wedge \mathit{F.Dead} \right)$$

There exists a path on which Frogger is standing on a diving-turtle group at the moment it enters the submerged phase, which causes the cell to become open water and the frog to drown. This checks the interaction between the **DivingTurtleCycle** and the **FloatingLane** through the `diving_turtles_submerged` channel.

Execution time: ∼15s

3.2 Stochastic Analysis

This section evaluates the stochastic variant introduced above through Statistical Model Checking, using the plain weighted policy defined in the design section for Frogger and the Vehicle Lane. The resulting estimates serve as a baseline: a later section repeats the same analysis with a more advanced, context-aware movement policy, to check whether it improves Frogger's chances.

3.2.1 Completing the first level with at least 200 points

Given the above specification for the stochastic version, simulate the game for a maximum of 1000 time units and estimate the probability of completing the first level while scoring at least 200 points.

Running `Pr[<=1000;1000](<> level==2 && score>=200)` on the stochastic model yields a probability estimate with a 99% confidence interval of $[0.000338, 0.010934]$, which is essentially negligible. This is a direct consequence of the frog's movement policy: the weighted branches introduced in the stochastic variant (0.5 UP, 0.2 RIGHT, 0.2 LEFT, 0.1 DOWN) have no awareness of the surrounding cells, so Frogger keeps moving with the same fixed bias regardless of an incoming vehicle, an empty water cell, or a nearby log. Crossing the lanes turned out to be more difficult for the frog than expected. The probability of Frogger reaching 200 points and completing the first level is close to zero: in order to improve the frog's chances, a more sophisticated movement policy is necessary.

3.2.2 Context-Aware Movement Policy

This section presents the context-aware movement policy implementation, and repeats the same Statistical Model Checking analysis to assess whether it improves Frogger's chances with respect to the fixed weighted policy. The policy combines a legality check, a one-step predictive layer, and a probability-weighted decision rule, so that Frogger's behaviour depends on the current and near-future state of the board.

Move legality. Before a direction is evaluated, the policy checks that Frogger's current position lies within the grid and that the target move cell lies within the grid: Frogger can never voluntarily step outside the playing field, unless it's carried out by a moving log. A candidate move is then discarded if the target cell is:

- a wall (`X`);
- an already-occupied home bay (`H`);
- a cell currently occupied by a vehicle (`V`);
- open water (`W`);
- a diving turtle that is currently submerged (`D`).

Predictive layer. Among the moves that pass the legality check, the policy additionally looks one step ahead in time: a move is discarded if the vehicle occupying the target lane will reach that cell on its next update, and likewise if the target cell is a diving turtle about to submerge.

Direction selection. Each surviving direction receives an integer weight between 0 and 5, based on the weighted probability assigned by the policy. Forward movement is favoured by default; weights are additionally increased for directions that reduce Frogger's distance to the nearest unoccupied home bay, biasing progress towards the closest available goal rather than an arbitrary one. Direction is then sampled stochastically according to these weights.

River behaviour. While Frogger is on the river and no adjacent floating object is currently safe to board, the policy departs from the general weighting rule and instead directs movement towards the nearest log or turtle and, simultaneously, the nearest free home bay.

No movement handling. If every direction has probability zero because no direction is safe, the *can't move* option is activated, allowing Frogger to remain in the same position instead of being forced to choose non-deterministically among unsafe moves. The *can't move* option is in mutual exclusion with the other four movement directions, so that Frogger either moves or stays still, but never both.

3.2.3 Context-Aware Policy Analysis

We simulated the policy with `Pr[<=10000; 1000](<>level==13)`, estimating, over 1000 runs bounded at 10000 time units, the probability that Frogger reaches level 13 obtaining the following results.

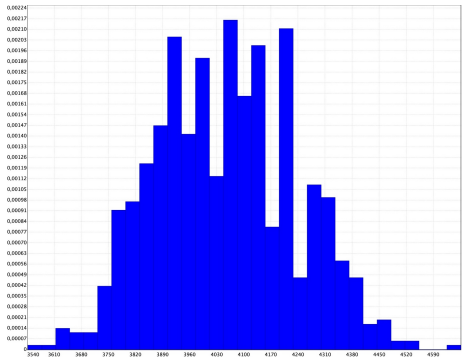


Figure 11: Probability Density Distribution of the stochastic enhanced model

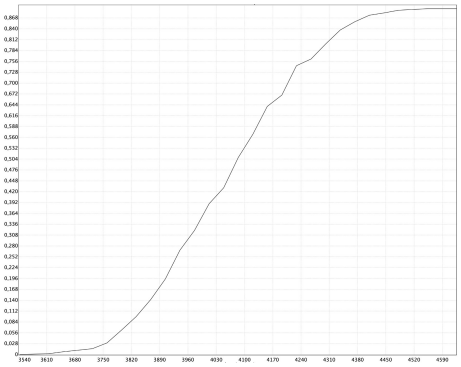


Figure 12: Cumulative Probability Distribution of the stochastic enhanced model

Level-completion probability. The Statistical Model Checking estimate for `Pr[<=10000; 1000](<>level>=13)`, shown in Figure 12, is 0.89 ± 0.03 (95% CI): the context-aware policy reaches level 13 or beyond, considering level 14 as the maximum reachable level, in roughly 89.18% of the 1000 runs, a substantial improvement over the fixed weighted policy.

Timing distribution. The density (Figure 11) shows that, when level 13 is reached, it happens predominantly between $t \approx 3900$ and $t \approx 4200$.

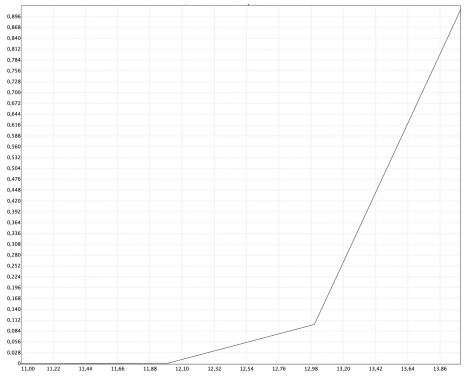


Figure 13: Cumulative Probability Distribution of the maximum level reached with the stochastic enhanced model

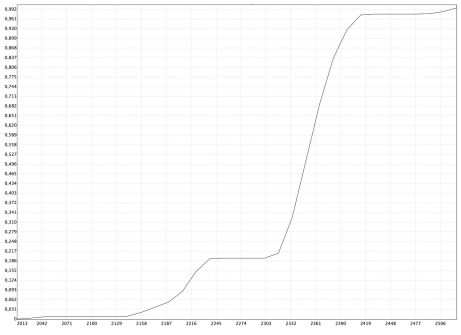


Figure 14: Cumulative Probability Distribution of the maximum score reached with the stochastic enhanced model

Levels completed. Figure 13 shows the cumulative distribution of the maximum level reached within each run (`E[<=8000;1000](max: level)`). The curve rises sharply and is concentrated in a narrow band around level 12–14, consistent with the estimated expected value of 12.98 ± 0.03

(95% CI): on average, the context-aware policy takes Frogger almost all the way to level 13 before the simulation bound is reached, with little variance across runs.

Score achieved. Figure 14 shows the cumulative distribution of the maximum score reached within each run (`E[<=8000;1000](max: score)`), with an estimated expected value of 2329 ± 5 (95% CI), consistent with the sustained progress described above.

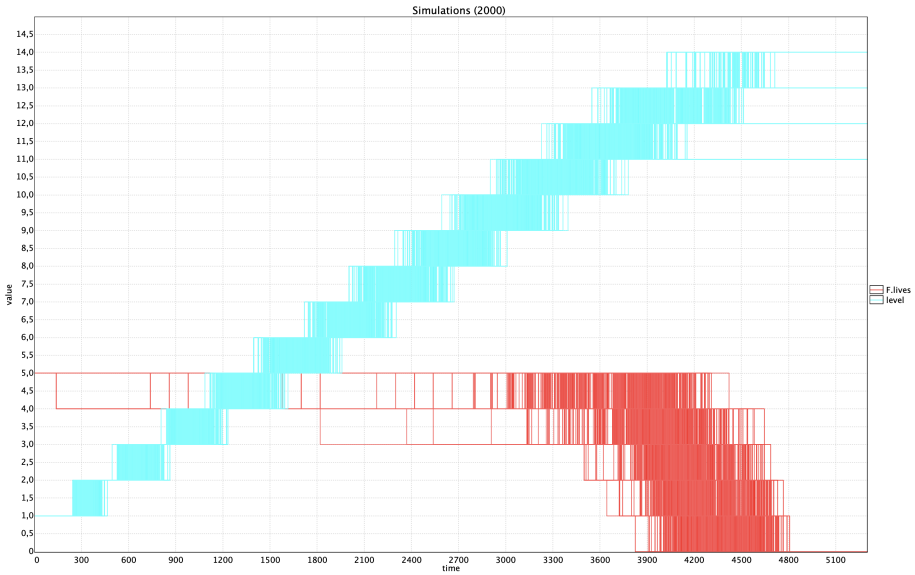


Figure 15: Distribution of `level` and `F.lives` over time across 2000 SMC simulation runs.

Level and lives progression. Figure 15 overlays the `level` and `F.lives` traces of 2000 simulation runs. Levels advance in discrete steps early on, with the spread widening at higher levels as variance accumulates. Most of the deaths are concentrated in the final stages, while the majority of runs still manage to reach level 14, the last reachable level; this confirms that the enhanced stochastic policy yields to optimum results, allowing Frogger to sustain progress through most of the game while maintaining most of its lives. Figure 15 further reveals a physical wall beyond which Frogger is unable to survive, indicating that this late-stage mortality does not stem from a deficiency in the movement policy itself, but from a hard constraint imposed by the game's design: once level 14 is reached, Frogger is invariably doomed to die shortly after respawning.

4 Conclusion

To summarize, we modelled and formally verified a simplified version of Frogger in UPPAAL, developing both a symbolic and a stochastic version of the system as a network of Timed Automata that captures all the core mechanics of the game. We verified a set of TCTL properties through the UPPAAL verifier, proving the reachability of requested queries. The work highlighted some design trade-offs adopted to avoid the state space explosion during verification, keeping the final model compact and efficient while preserving a faithful representation of the game logic.